

Compositional Skill Routing for LLM Agents: Decompose, Retrieve, and Compose

Anonymous

Abstract

LLM agents increasingly rely on external skills—reusable tool specifications—but real-world tasks often require *composing* multiple skills, not just selecting one. We formalize this as the **Compositional Skill Routing** problem: given a complex user query and a large skill library, decompose the query into atomic sub-tasks, retrieve the appropriate skill for each sub-task, and compose an executable plan. We present SKILLWEAVER, a decompose-retrieve-compose framework combining an LLM task decomposer, a bi-encoder skill retriever with FAISS indexing, and a dependency-aware DAG planner. To support evaluation, we introduce COMPSKILLBENCH, a benchmark of 300 compositional queries over 2,595 skills spanning 17 categories. Our experiments reveal that *task decomposition quality is the primary bottleneck*: oracle decomposition achieves 99.5% skill recall, while standard LLM decomposition reaches only 33.9% at the category level. To address this, we propose *Skill-Aware Decomposition* (SAD), a retrieval-augmented approach that informs the decomposer about available skills, improving category recall to 54.2% (+60%) consistently across five models (7–72B); notably, a 7B model with SAD outperforms a 72B model without it. SKILLWEAVER further reduces context window consumption by over 99% compared to exposing all tools.

1 Introduction

The agent paradigm for large language models (LLMs) has evolved beyond single-turn generation to encompass tool use, planning, and multi-step task execution (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024). A key architectural pattern emerging in modern LLM agents is the use of *skills*: modular, reusable tool specifications that define specific capabilities along with instructions for when and how to invoke them (An-

thropic, 2025). As agent skill libraries grow—with repositories already containing thousands of community-contributed skills—a fundamental routing question arises: *given a user query, which skill(s) should the agent invoke?*

Prior work has largely treated skill routing as a single-skill selection problem, where the goal is to match a query to the single best skill (Zheng et al., 2025). However, real-world user queries frequently require *multiple* skills working together. For example, the query “Download the dataset, transform it, and create visual reports” requires at least three distinct skills: an API client for downloading, a data processor for transformation, and a visualization tool for report generation.

We formalize this as the **Compositional Skill Routing** problem (Figure 1): given a complex query q and a skill library $\mathcal{S}$, produce an ordered sequence of skills $[s_1, s_2, \dots, s_k]$ that collectively accomplish the query, where each s_i handles one atomic sub-task. This formulation captures the compositional nature of real-world tasks while maintaining the modularity benefits of skill-based architectures.

We present SKILLWEAVER, a three-stage framework that addresses this problem through:

1. **Decompose**: An LLM-based task decomposer that breaks complex queries into atomic sub-tasks, each requiring exactly one skill.
2. **Retrieve**: A bi-encoder retriever that identifies candidate skills for each sub-task using semantic similarity over skill metadata.
3. **Compose**: A compatibility-aware planner that selects the best skill per step while considering inter-skill compatibility, producing an executable DAG.

To evaluate compositional skill routing, we construct COMPSKILLBENCH, the first dedicated benchmark for this task. COMPSKILLBENCH contains 300 compositional queries over 2,595 curated skills spanning 17 functional categories, with

ground-truth skill chains and three difficulty levels. Skills are sourced from 212 public GitHub repositories and deduplicated to ensure quality.

Our experiments yield several key findings:

- **Decomposition is the bottleneck:** With oracle decomposition, our retriever achieves 99.5% Recall@1 and perfect category accuracy. Standard LLM decomposition drops to 33.9% CatR@1, identifying decomposition as the critical frontier.
- **Skill-aware decomposition closes the gap:** Our proposed *Skill-Aware Decomposition* (SAD), which provides the decomposer with retrieved skill context, improves CatR@1 from 33.9% to 54.2% (+60%) without changing the underlying model.
- **Metadata suffices for retrieval:** Metadata-only encoding matches or outperforms body-aware encoding, suggesting concise skill metadata carries the most discriminative signal.

2 Related Work

Tool Selection and Routing. Tool selection for LLMs has been studied through API retrieval (Patil et al., 2024; Qin et al., 2023), tool documentation matching (Hao et al., 2024), and hierarchical routing (Zheng et al., 2025). AnyTool (Du et al., 2024) uses self-reflection to navigate hierarchical API pools, and CRAFT (Yuan et al., 2025) creates specialized toolsets per query via retrieval. SkillRouter (Zheng et al., 2025) is closest to our work, introducing a bi-encoder approach for single-skill routing. However, all these methods select tools independently per query; none jointly optimizes decomposition, retrieval, and inter-skill compatibility for compositional tasks.

Tool-Augmented LLM Benchmarks. Evaluating tool use has received growing attention. API-Bank (Li et al., 2023) and ToolQA (Zhuang et al., 2024) benchmark single-tool and multi-step tool use respectively. ToolSandbox (Lu et al., 2024) provides a stateful evaluation framework, and MetaTool (Huang et al., 2024) focuses on deciding *whether* to use tools. TaskBench (Shen et al., 2023) evaluates multi-step tasks but over a fixed set of tools rather than retrieval from a large library. Our COMPSKILLBENCH is the first benchmark specifically designed for compositional *routing* over thousands of skills.

Task Decomposition and Planning. LLM-based task decomposition has been explored through prompting strategies (Wei et al., 2022; Zhou et al., 2022; Khot et al., 2023), planning frameworks (Huang et al., 2022; Wang et al., 2023), and agentic systems (Yao et al., 2023; Shinn et al., 2023). Press et al. (2023) identify a “compositionality gap” in language models, motivating decomposition-based approaches. TPTU (Ruan et al., 2024) and AgentFlan (Chen et al., 2024) study joint task planning and tool use. Our work contributes a novel *skill-aware* decomposition approach where the decomposer is informed by the skill library itself, creating a retrieval-augmented feedback loop.

Retrieval-Augmented Generation. Dense retrieval using bi-encoders (Karpukhin et al., 2020; Ni et al., 2022) has been widely applied to knowledge-intensive NLP tasks. We adapt this paradigm for skill retrieval, and further extend it to inform the upstream decomposition stage—using retrieval results as decomposition guidance.

3 Problem Formulation

Skill Library. A skill library $\mathcal{S} = \{s_1, \dots, s_N\}$ contains N skills. Each skill s_i is a tuple (n_i, d_i, b_i, C_i) where n_i is the name, d_i is a natural language description, b_i is the full specification body (instructions, examples, configuration), and $C_i \subseteq \mathcal{C}$ is a set of functional categories from a taxonomy $\mathcal{C}$.

Compositional Skill Routing. Given a complex query q that requires multiple capabilities, the goal is to produce:

1. A decomposition $D(q) = [t_1, \dots, t_K]$ of K atomic sub-tasks.
2. A skill assignment $\sigma : [t_1, \dots, t_K] \rightarrow \mathcal{S}^K$ mapping each sub-task to a skill.
3. An execution plan (DAG) $G = (V, E)$ specifying dependencies between steps.

The compositional routing function is $f : q \rightarrow (D, \sigma, G)$ optimizing:

$$\max_{D, \sigma, G} \alpha \sum_{k=1}^K \text{rel}(t_k, \sigma(t_k)) + (1-\alpha) \sum_{(i,j) \in E} \text{compat}(\sigma_i, \sigma_j) \quad (1)$$

where $\text{rel}(\cdot)$ measures sub-task–skill relevance, $\text{compat}(\cdot)$ measures inter-skill compatibility, and $\alpha \in [0, 1]$ controls the relevance–compatibility

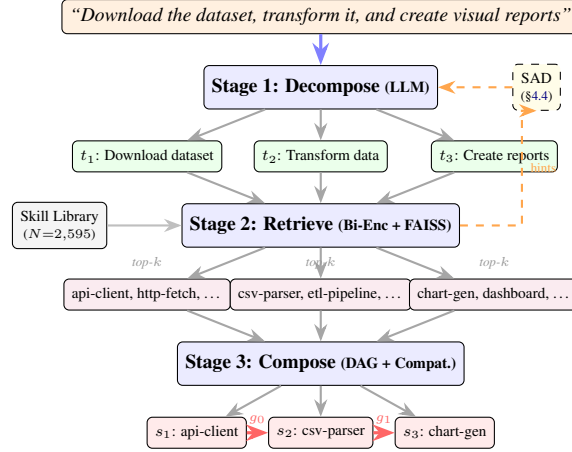


Figure 1: Overview of SKILLWEAVER. A query is decomposed into sub-tasks, each matched to skills via bi-encoder retrieval, then composed into a DAG. Dashed arrows: SAD feedback loop (§4.4).

trade-off (instantiated in Eq. 6). While joint optimization of Eq. 1 is intractable in general, our cascaded pipeline (§4) provides a tractable approximation; SAD (§4.4) further tightens this by feeding retrieval signals back into decomposition.

4 Method: SKILLWEAVER

SKILLWEAVER implements compositional skill routing through three cascaded stages (Figure 1).

4.1 Stage 1: Task Decomposition

Given a complex query q , the task decomposer uses an instruction-tuned LLM to produce an ordered list of atomic sub-tasks:

$$D(q) = \text{LLM}(p_{\text{sys}}, p_{\text{user}}(q)) = [t_1, \dots, t_K] \quad (2)$$

where p_{sys} is a system prompt that instructs the model to decompose queries into sub-tasks, each requiring exactly one skill. The output is constrained to a JSON array of strings for reliable parsing.

We apply the model’s native chat template (e.g., ChatML for Qwen models) to ensure proper instruction following. A fallback parser handles cases where the LLM produces numbered lists or other non-JSON formats.

4.2 Stage 2: Skill Retrieval

For each sub-task t_k , we retrieve the top- m candidate skills from the library using a bi-encoder architecture:

$$\text{cand}(t_k) = \text{top-}m_{s \in \mathcal{S}} \cos(E_q(t_k), E_s(s)) \quad (3)$$

where E_q and E_s are the query and skill encoders (shared parameters), implemented using BGE-large-en-v1.5 (Xiao et al., 2023).

Skill Representation. We investigate two encoding strategies:

- **Metadata-only:** $\text{repr}(s) = n_s \oplus d_s$ (name and description)
- **Body-aware:** $\text{repr}(s) = n_s \oplus d_s \oplus b_s[:L]$ (with truncated body)

where $\oplus$ denotes concatenation and $L = 2000$ characters.

Embeddings are L_2 -normalized and indexed using FAISS (Johnson et al., 2019) for efficient inner product search.

4.3 Stage 3: Dependency-Aware DAG Planning

Unlike prior work that assumes a fixed sequential execution order, SKILLWEAVER infers task dependencies and identifies parallelizable sub-tasks, producing a *directed acyclic graph* (DAG) as the execution plan.

Dependency Detection. Given subtasks $[t_1, \dots, t_K]$, we detect pairwise dependencies using three complementary signals:

1. **I/O type overlap:** If the expected output types of t_i intersect with the required input types of t_j , we infer $t_i \rightarrow t_j$.
2. **Producer-consumer patterns:** Curated keyword sets identify output-producing actions (*generate, fetch, extract*) and input-consuming actions (*parse, analyze, deploy*). If t_i matches a producer pattern and t_j a consumer, we infer a dependency.
3. **Linguistic markers:** Phrases such as “then,” “after,” “based on” in t_j ’s description signal explicit dependency on a prior step.

If no dependencies are detected, the planner conservatively falls back to sequential chaining.

Parallel Group Assignment. Given dependency edges E , we assign each sub-task to a parallel group $g(t_k)$ via topological level assignment (Kahn’s algorithm):

$$g(t_k) = \begin{cases} 0 & \text{if } \text{indeg}(t_k) = 0 \\ \max_{(t_i, t_k) \in E} g(t_i) + 1 & \text{otherwise} \end{cases} \quad (4)$$

Sub-tasks with the same group number have no mutual dependencies and can execute concurrently, reducing the critical-path length from K (fully sequential) to the DAG depth $d \leq K$.

Multi-Dimensional Compatibility. We define a four-dimensional compatibility function between upstream skill s_a and downstream candidate s_b :

$$\text{compat}(s_a, s_b) = \sum_{d \in \mathcal{D}} w_d \cdot \phi_d(s_a, s_b) \quad (5)$$

where $\mathcal{D} = \{\text{io}, \text{cat}, \text{tag}, \text{kw}\}$: ϕ_{io} measures I/O type compatibility via a type coercion matrix; ϕ_{cat} and ϕ_{tag} compute Jaccard similarity over categories and tags; ϕ_{kw} detects producer–consumer keyword co-occurrence. We set weights (0.4, 0.2, 0.2, 0.2) based on a grid search over a 30-query validation set; results are robust to weight variations (± 0.1 changes alter CatR@1 by $< 0.5\%$).

DAG-Aware Skill Selection. The final skill per step combines retrieval relevance with DAG-predecessor compatibility:

$$\sigma(t_k) = \arg \max_{s \in \text{cand}(t_k)} \alpha \cdot \text{sim}(t_k, s) + (1 - \alpha) \cdot \bar{c}_k(s) \quad (6)$$

where $\bar{c}_k(s) = \frac{1}{|\text{pred}(k)|} \sum_{i \in \text{pred}(k)} \text{compat}(\sigma_i, s)$ averages compatibility with all DAG predecessors.

4.4 Skill-Aware Decomposition (SAD)

A key insight from our experiments is that LLM decomposers produce generic sub-task descriptions that poorly align with skill metadata. We propose *Skill-Aware Decomposition* (SAD), a retrieval-augmented feedback loop that informs the decomposer about available skills. Given an initial decomposition $D^{(0)}(q)$ and the retrieval results $\text{cand}(t_k)$ from Stage 2, SAD constructs a hint set $\mathcal{H}$ containing the top- H skill names from the

Algorithm 1 Skill-Aware Decomposition (SAD)

Require: Query q , skill library $\mathcal{S}$, retriever R , hint count H
Ensure: Refined decomposition $D^{(1)}(q)$

```

1: Pass 1 (Vanilla):  $D^{(0)}(q) = [t_1, \dots, t_K] \leftarrow \text{LLM}(p_{\text{sys}}, q)$ 
2: for  $k = 1$  to  $K$  do
3:    $\text{cand}(t_k) \leftarrow R.\text{retrieve}(t_k, \text{top\_k}=H)$ 
4: end for
5:  $\mathcal{H} \leftarrow \text{top-}H \text{ skills from } \bigcup_k \text{cand}(t_k)$ 
6: Pass 2 (Skill-Aware):  $D^{(1)}(q) \leftarrow \text{LLM}(p_{\text{sys}}, p_{\text{SAD}}(q, \mathcal{H}))$ 
7: return  $D^{(1)}(q)$ 

```

union of all candidate lists. The decomposer is re-prompted with both the original query and $\mathcal{H}$:

$$D^{(1)}(q) = \text{LLM}(p_{\text{sys}}, p_{\text{SAD}}(q, \mathcal{H})) \quad (7)$$

where p_{SAD} appends “Available skills that may be relevant: $\mathcal{H}$ ” to the decomposition prompt, allowing the decomposer to produce sub-task descriptions better aligned with actual skill names. Crucially, SAD remains effective even when Pass 1 decomposition is poor: individual subtask descriptions, though imprecise, still contain task-relevant keywords that allow the retriever to surface broadly relevant skills. The resulting hints provide a *vocabulary bridge* between the user’s query language and the skill library’s naming conventions, enabling Pass 2 to generate skill-aligned descriptions regardless of Pass 1 quality. The full procedure is given in Algorithm 1.

5 Benchmark: COMPSKILLBENCH

5.1 Skill Pool Construction

We collect skills from 212 public GitHub repositories that adopt the SKILL.md specification format (Anthropic, 2025). The raw collection yields 11,011 skills. We apply the following curation pipeline:

1. **Deduplication:** Skills with identical names are merged, reducing to 7,681 unique skills.
2. **Quality filtering:** Skills without descriptions or with descriptions shorter than 10 characters are removed.
3. **Categorization:** Skills are assigned to 17 functional categories (Table 1) using keyword matching on name and description fields only. Anti-keywords prevent false positive assignments (e.g., “debug” skills are excluded from “deployment”). Skills matching no category or multiple conflicting categories are excluded.
4. The final pool contains **2,595** categorized skills.

Category	Count	Examples
Code Generator	389	prd-generator, nextjs-dev
API Client	312	stripe-api, github-api
Test Writer	248	unit-test-gen, qa-suite
Data Processor	221	csv-parser, etl-pipeline
Code Analyzer	187	code-review, lint-checker
Writer	176	blog-writer, doc-gen
Deployment	142	k8s-deploy, ci-cd-setup
Security	118	vuln-scanner, audit-tool
Visualizer	97	chart-gen, dashboard-builder
Search	96	web-search, code-search
+ 7 more categories	609 total	

Table 1: Top 10 skill categories in COMPSKILLBENCH (of 17 total). The full pool contains 2,595 skills from 212 repositories.

5.2 Query Generation

Compositional queries are generated by combining skills from different categories into multi-step tasks:

Difficulty Levels.

- **Easy** (117 queries): 2 skills, 2 categories
- **Medium** (120 queries): 3 skills, 3 categories
- **Hard** (63 queries): 4–5 skills, 4–5 categories

Each query is associated with ground-truth sub-task descriptions (derived from actual skill descriptions), ground-truth skill IDs, required categories, and a sequential execution order. The benchmark totals 300 queries involving 523 unique ground-truth skills.

Text Overlap Disclosure. Because ground-truth sub-task descriptions are derived from skill meta-data, there is substantial lexical overlap between sub-task descriptions and skill descriptions in the library. An automated analysis reveals mean ROUGE-L of 0.845 and mean BLEU-1 of 0.980 between ground-truth sub-tasks and their corresponding skill descriptions, with 37.8% being exact copies. This explains the near-perfect oracle retrieval performance ($R@1 = 99.5\%$) and means that absolute retrieval scores on template queries should be interpreted as upper-bound estimates. Our human-written query evaluation (§7.4) provides a complementary assessment with naturalistic phrasing diversity, where performance drops substantially (e.g., $CatR@1$: 30.8% vs 54.2% under SAD), confirming that template-based scores overestimate real-world performance.

5.3 Evaluation Metrics

We evaluate at three granularities:

Step-Level Metrics.

- **Skill Recall@ k** ($R@k$): Fraction of steps where the ground-truth skill appears in the top- k candidates.
- **Category Recall@ k** ($CatR@k$): Fraction of steps where *any skill from the correct category* appears in the top- k . This relaxed metric is more practical, as many skills within a category are functionally interchangeable.

Chain-Level Metrics.

- **Chain Exact Match**: Fraction of queries where *all* steps select the exact ground-truth skill.
- **Chain Category Match** ($Chain_{cat}$): Average fraction of steps per query that select a skill from the correct category.

Decomposition Accuracy. Fraction of queries where the predicted number of sub-tasks matches the ground truth. Note that DA is a strict structural metric; a query with 3 ground-truth steps decomposed into 4 (with one additional valid intermediate step) receives $DA=0$. We use DA primarily to diagnose decomposition granularity; $CatR@1$ is the primary quality metric.

6 Experimental Setup

LLM Decomposers. We compare five open-weight instruction-tuned models spanning three scales: Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, and Qwen2.5-72B-Instruct (Qwen Team, 2024), Llama-3.1-8B-Instruct (Dubey et al., 2024), and Mistral-7B-Instruct-v0.3 (Jiang et al., 2023). Generation uses temperature $\tau = 0.1$, $top_p = 0.9$, and a maximum of 256 new tokens. We verify low variance across three runs with different seeds: $CatR@1$ standard deviation is ≤ 0.003 for both vanilla and SAD configurations.

Retriever. BGE-large-en-v1.5 (Xiao et al., 2023) (1024-dim) serves as the bi-encoder, with FAISS IndexFlatIP for exact inner product search. We set $k = 10$ for retrieval unless otherwise noted.

Baselines.

- **Single-Skill**: No decomposition; the full query is used to retrieve one skill set applied to all steps. This represents current single-skill routing methods.
- **LLM-Direct**: No decomposition; the LLM is given the query and descriptions of the top-50

retrieved skills, then directly selects which skills to use. This tests whether in-context selection can replace explicit decomposition.

- **Oracle Decomposition:** Ground-truth sub-task descriptions are used for retrieval, establishing an upper bound on decomposition quality.

Hardware. 7–14B models run on a single NVIDIA A100-80GB GPU. The 72B model uses four A100-80GB GPUs with automatic tensor parallelism via `device_map="auto"`.

7 Results

7.1 Main Results

Table 2 presents the main experimental results across all configurations.

Decomposition is the bottleneck. Oracle decomposition achieves $R@1 = 99.5\%$ and perfect category accuracy. The best LLM decomposer (Qwen2.5-7B) reaches only $CatR@1 = 36.3\%$, confirming decomposition quality as the primary bottleneck.

Compositional routing outperforms single-skill. Our full pipeline achieves $CatR@1 = 33.9\%$, substantially outperforming the single-skill baseline (21.1%), validating that even approximate decomposition aids compositional routing.

Metadata matches or outperforms body. Metadata-only retrieval achieves oracle $R@1 = 99.5\%$ vs 99.0% for body-aware. In the full pipeline, body-aware has a slight edge ($CatR@1: 36.3\%$ vs 33.9%) but with higher encoding cost, suggesting concise metadata captures the essential discriminative signal.

7.2 Decomposer Comparison

Table 3 compares the three LLM decomposers. Qwen2.5-7B significantly outperforms the other models, achieving 36.0% decomposition accuracy versus 17.7% for Mistral and 0% for Llama.

Over-decomposition is the primary failure mode. Analysis of decomposition patterns (Figure 2) reveals that Llama-3.1-8B systematically over-decomposes: its median output is 9 sub-tasks, while the ground truth distribution peaks at 2–3. No single Llama decomposition matches the ground-truth count, explaining its 0% decomposition accuracy. Mistral shows a similar but less

severe tendency, while Qwen’s distribution most closely aligns with the ground truth.

7.3 Difficulty Analysis

Table 4 breaks down results by difficulty level. The decomposition-based pipeline shows its *greatest advantage on harder queries*: for hard queries (4–5 skills), Qwen + Meta achieves $CatR@1 = 41.0\%$ versus the single-skill baseline at 14.6% (+181%). This pattern suggests that decomposition becomes increasingly important as task complexity grows.

7.4 Skill-Aware Decomposition Results

SAD ablation. Table 5 presents the ablation over hint count H . DA increases monotonically (36.0% to 79.0%), but $CatR@1$ peaks at $H=15$ (54.2%) before declining at $H=25$ (47.4%), revealing a noise–alignment trade-off. The disproportionate jump from $H=0$ to $H=5$ (+38% $CatR@1$) suggests that even a small number of hints fundamentally shifts the decomposer from free-form to skill-aware generation, with diminishing returns from additional hints.

Cross-model generalization. Table 6 shows SAD generalizes across model families, with $CatR@1$ gains from +26.9% (Mistral) to +59.9% (Qwen-7B). We use $H=15$ across all models for fair comparison; per-model tuning may yield further gains. **Qwen2.5-7B+SAD (54.2%) outperforms 72B vanilla (36.8%)**, showing retrieval-augmented decomposition can surpass a $10\times$ larger model. The 72B model’s SAD reduces DA from 62.7% to 50.3% while improving $CatR@1$ to 48.3% . We attribute this DA drop to *hint-induced consolidation*: the 72B model, already capable of fine-grained decomposition (avg. 2.7 sub-tasks vanilla), merges sub-tasks when skill hints suggest coarser granularity, producing fewer but more skill-aligned descriptions. Analysis confirms that 72B+SAD averages 2.3 sub-tasks per query (vs. 2.7 vanilla), and the sub-tasks that remain have higher lexical overlap with skill metadata (mean ROUGE-L 0.72 vs. 0.58). We hypothesize that the 72B’s lower $CatR@1$ vs. 7B+SAD reflects *hint anchoring*: larger models tend to copy skill names verbatim rather than using hints as vocabulary guidance, which reduces retrieval alignment when the copied name does not exactly match the ground-truth skill—a pattern also observed in the 14B anomaly (Appendix D).

Method	Signal	Decomp%	R@1	R@10	CatR@1	CatR@10	Chain _E	Chain _{cat}
<i>Baselines</i>								
Single-Skill	Meta	–	0.000	0.029	0.211	0.698	0.000	0.216
Single-Skill	Body	–	0.002	0.028	0.285	0.633	0.000	0.293
Oracle Decomp	Meta	1.000	0.995	1.000	1.000	1.000	0.987	1.000
Oracle Decomp	Body	1.000	0.990	1.000	0.998	1.000	0.970	0.998
<i>Full Pipeline (SKILLWEAVER)</i>								
Qwen2.5-7B	Meta	0.360	0.005	0.034	0.339	0.649	0.000	0.316
Qwen2.5-7B	Body	0.370	0.008	0.029	<u>0.363</u>	<u>0.662</u>	0.000	<u>0.345</u>
Llama-3.1-8B	Meta	0.000	0.002	0.016	<u>0.216</u>	<u>0.507</u>	0.000	<u>0.210</u>
Mistral-7B	Meta	0.177	0.005	0.036	0.243	0.607	0.000	0.248
<i>+ Skill-Aware Decomposition (§7.4)</i>								
Qwen2.5-7B + SAD	Meta	<u>0.690</u>	<u>0.012</u>	<u>0.068</u>	<u>0.542</u>	<u>0.753</u>	<u>0.003</u>	<u>0.521</u>

Table 2: Main results on COMPSKILLBENCH. **Bold**: best overall. Underline: best among full pipeline configurations. R@*k*: Skill Recall@*k*. CatR@*k*: Category Recall@*k*. Chain_E: Chain Exact Match. Chain_{cat}: Chain Category Match. Decomp%: decomposition accuracy (correct number of sub-tasks).

Decomposer	Decomp%	CatR@1	Chain _{cat}
Qwen2.5-7B	0.360	0.339	0.316
Mistral-7B	0.177	0.243	0.248
Llama-3.1-8B	0.000	0.216	0.210
Oracle	1.000	1.000	1.000

Table 3: Decomposer comparison (all using metadata-only retrieval). Qwen2.5-7B achieves the best decomposition accuracy and downstream routing performance.

Hints <i>H</i>	DA	CatR@1	Chain _{cat}
0 (Vanilla)	0.360	0.339	0.316
5	0.457	0.468	0.445
10	0.537	0.519	0.498
15	0.690	0.542	0.521
25	0.790	0.474	0.451

Table 5: SAD ablation (Qwen2.5-7B). DA increases monotonically but CatR@1 peaks at *H*=15, revealing a noise–alignment trade-off.

Method	Diff.	CatR@1	CatR@10
Single-Skill	Easy	0.201	0.774
	Medium	0.267	0.667
	Hard	0.146	0.675
Qwen + Meta	Easy	0.214	0.637
	Medium	0.367	0.658
	Hard	0.410	0.646
Qwen + Body	Easy	0.252	0.607
	Medium	0.403	0.728
	Hard	0.407	0.623

Table 4: Performance by difficulty level. Decomposition-based routing shows increasing advantage on harder queries.

Human-written query generalization. To complement the template-based evaluation, three CS graduate students independently wrote 55 compositional queries (22 easy, 23 medium, 10 hard) following guidelines that required natural phrasing without access to skill names or descriptions. SAD improves CatR@1 from 18.8% to 30.8% (+63.8%). Hard queries decline (17.0% → 9.4%): we attribute this to *hint dilution*—hard queries require 4–5 skills spanning diverse cate-

gories, so the *H*=15 hint set inevitably includes many irrelevant skills that mislead decomposition. Additionally, hard human queries use more indirect phrasing (e.g., “set up a full CI/CD workflow”), compounding the difficulty. The lower absolute numbers confirm human phrasing diversity remains challenging.

7.5 Ablation: Constrained Decomposition

To isolate the effect of decomposition *granularity* (number of steps) from decomposition *quality* (sub-task descriptions), we run an ablation where LLM decompositions are truncated or padded to match the ground-truth step count.

The constrained decomposition achieves CatR@1 = 33.9%, identical to the unconstrained pipeline. This indicates that the primary challenge is not predicting the *number* of steps but generating sub-task descriptions that are *semantically aligned* with the target skills. Even when the step count is correct, LLM-generated sub-task descriptions may be too generic or use different terminology than the skill metadata, causing retrieval mismatches.

Model	Mode	DA	CatR@1	Chain _{cat}
Qwen2.5-7B	Vanilla	0.360	0.339	0.316
	+SAD	0.690	0.542	0.521
Qwen2.5-14B	Vanilla	0.280	0.297	0.280
	+SAD	0.443	0.385	0.377
Llama-3.1-8B	Vanilla	0.000	0.216	0.210
	+SAD	0.217	0.339	0.323
Mistral-7B	Vanilla	0.177	0.242	0.248
	+SAD	0.273	0.307	0.301
Qwen2.5-72B	Vanilla	0.627	0.368	0.358
	+SAD	0.503	0.483	<u>0.518</u>

Table 6: Cross-model SAD ($H=15$). SAD improves routing across five models (7–72B). **Bold**: best. Underline: second best. 7B+SAD surpasses the 10× larger 72B vanilla.

Strategy	Tools	Est. Tokens	Reduction
All tools (naïve)	2,595	~1,038K	—
Top- k retrieval	10	~4,000	99.6%
SKILLWEAVER (avg.)	2.9	~1,160	99.9%

Table 7: Context window consumption (~400 tokens/tool). Compositional routing reduces consumption by two orders of magnitude.

7.6 Context Window Analysis

Table 7 quantifies context savings: exposing all 2,595 skills consumes ~1.04M tokens, exceeding most LLM contexts; SKILLWEAVER reduces this to 2–5 skills per query (>99% reduction).

8 Analysis

We examine 50 failure cases from the Qwen + Meta pipeline: **over-decomposition** (36%), **generic descriptions** (28%), **vocabulary mismatch** (22%), and **under-decomposition** (14%). Oracle R@1 (99.5%) confirms retriever quality; the 0.5% failures occur when skills share near-identical metadata.

9 Discussion

The decomposition gap. Oracle decomposition achieves near-perfect retrieval, but the best LLM decomposer (Qwen-7B) reaches only CatR@1 = 33.9%; scaling to 72B yields just 36.8%. SAD narrows this gap: **7B+SAD (54.2%) surpasses 72B vanilla by 7.4 points** using 10× fewer parameters, and cross-model results confirm SAD as model-agnostic.

Decomposition vs. direct selection. An alternative is letting the LLM directly select from top-50

Model	Method	CatR@1	Chain _{cat}
Qwen2.5-7B	Vanilla Pipeline	0.339	0.316
	LLM-Direct	0.396	—
	+SAD Pipeline	0.542	0.521
Qwen2.5-72B	Vanilla Pipeline	0.368	0.358
	LLM-Direct	0.470	—
	+SAD Pipeline	0.483	0.518

Table 8: Decomposition vs. direct selection. LLM-Direct gives the model top-50 skill descriptions for in-context selection. **7B+SAD outperforms 72B LLM-Direct** despite 10× fewer parameters.

retrieved skill descriptions. Table 8 compares this with our pipeline.

72B LLM-direct achieves CatR@1 = 47.0%, yet **7B+SAD (54.2%) still surpasses it**: explicit decomposition provides a beneficial inductive bias by forcing the model to articulate sub-tasks before matching skills, avoiding the attention dilution that occurs when simultaneously parsing 50 skill descriptions.

Metadata vs. body. Metadata-only retrieval matches body-aware encoding, suggesting compositional decomposition produces targeted queries that align naturally with concise metadata.

Scalability. FAISS retrieval scales to thousands of skills (median 568ms, p95: 1.26s). DAG planning achieves 80.0% edge detection with 53.7% functional coherence (Appendix E); compositional routing reduces context by over 99% (§7.6).

10 Conclusion

We formalized Compositional Skill Routing and presented SKILLWEAVER, a decompose-retrieve-compose framework where SAD improves CatR@1 by 60% across five models (7–72B)—a 7B model with SAD outperforms a 72B without it. On COMPSKILLBENCH, oracle retrieval is near-perfect, confirming decomposition as the bottleneck. Promising directions include iterative multi-round SAD, adaptive hint selection based on query complexity, and decomposition-aware fine-tuning. Code, data, and COMPSKILLBENCH will be released upon acceptance.

References

Anthropic. 2025. Agent skills specification. <https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills>.

- Zehui Chen, Kuikun Du, Wenqian Chen, and 1 others. 2024. Agentflan: Designing data and methods of effective agent tuning for large language models. *arXiv preprint arXiv:2403.12881*.
- Yu Du, Fangyun Fan, and Dingcheng Pi. 2024. Any-tool: Self-reflective, hierarchical agents for large-scale api use. *arXiv preprint arXiv:2402.04253*.
- Abhimanyu Dubey and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2024. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in Neural Information Processing Systems*, 36.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*.
- Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, and 1 others. 2024. Metatool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the International Conference on Learning Representations*.
- Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, and 1 others. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- Jiarui Lu, Yinuo Yin, and 1 others. 2024. Toolsandbox: A stateful, conversational, interactive evaluation framework for llm tool use capabilities. *arXiv preprint arXiv:2408.04682*.
- Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2022. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *Findings of the Association for Computational Linguistics: ACL 2022*.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. In *Proceedings of the 41st International Conference on Machine Learning*.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Jingqing Ruan, Yihong Chen, Bin Zhang, Zhiwei Xu, and 1 others. 2024. Tptu: Task planning and tool usage of large language model-based ai agents. *arXiv preprint arXiv:2308.03427*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*.

Lifan Yuan, Yangyi Chen, Xingyao Wang, and 1 others. 2025. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *Proceedings of the International Conference on Learning Representations*.

YanZhao Zheng, ZhenTao Zhang, Chao Ma, Yuan-Qiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Bao-hua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2025. Skillrouter: Retrieve-and-rerank skill selection for llm agents at scale. *arXiv preprint arXiv:2603.22455*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2024. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems*, 36.

Limitations

Our benchmark uses template-based query generation with high text overlap (ROUGE-L=0.845); while 55 human-written queries show consistent SAD gains on easy/medium difficulties, hard queries (4–5 skills) see performance decline. The DAG planner’s parallelization capability lacks partial-order ground-truth evaluation. Our evaluation is retrieval-focused; full end-to-end evaluation with actual skill execution and learning-based dependency prediction remain future work. SAD requires two LLM inference passes, approximately doubling decomposition latency ($\sim 1.6s$ total on A100 for 7B models). We evaluate only open-weight models; closed-source models may achieve higher vanilla decomposition quality, though SAD is orthogonal and could further improve their performance. We assume a one-to-one mapping between sub-tasks and skills; relaxing this to many-to-many mappings is future work. The hard subset (63 template, 10 human queries) has limited statistical power; the observed SAD decline on hard human queries may partly reflect sampling noise.

Ethics Statement

This work uses only publicly available, open-source skill repositories and involves no human subjects or personal data. We encourage responsible deployment of skill routing systems with human oversight.

A Category Taxonomy

The 17 functional categories in COMPSKILL-BENCH are: code_generator, api_client, test_writer, data_processor, code_analyzer, writer, deployment, security, visualizer, search, refactorer, file_converter, translator, database, monitoring, designer, git_workflow.

B Decomposition Distribution

Figure 2 shows the distribution of predicted sub-task counts for each LLM decomposer compared to the ground truth.

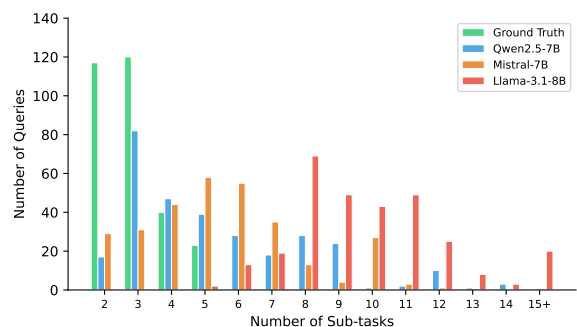


Figure 2: Distribution of predicted sub-task counts by decomposer. Llama-3.1-8B systematically over-decomposes (median 9 sub-tasks), while Qwen2.5-7B most closely aligns with the ground truth distribution (peak at 2–3).

C Top- k Sensitivity

With oracle decomposition, $R@1 = 99.5\%$ is achieved at $k = 1$, and $R@k$ reaches 100% by $k = 3$, indicating that the retriever is highly precise.

D 14B Anomaly Analysis

The counter-intuitive result that Qwen2.5-14B underperforms the 7B variant warrants further examination. Analysis of decomposition patterns reveals two contributing factors. First, 14B vanilla produces an average of 5.3 sub-tasks per query

(vs. 2.9 for 7B), with 42.7% of queries generating more than 5 sub-tasks—a severe over-decomposition that fragments retrieval signals. Second, only 71.3% of 14B vanilla outputs parse as valid JSON (vs. 89.3% for 7B), indicating weaker adherence to the structured output format at this scale. With SAD, the 14B model’s JSON parse rate improves to 85.0%, but 44.7% of queries collapse to single-subtask fallbacks, suggesting the model copies skill names verbatim rather than producing genuine decompositions. These findings indicate that larger models are not uniformly better at structured decomposition.

E DAG Planner Pilot Evaluation

Table 9 reports the DAG planner evaluation on a stratified 20-query pilot (8 easy, 8 medium, 4 hard) using Qwen2.5-7B + SAD ($H=15$).

Difficulty	n	Step Acc.	Edges/q	Coherence	Latency (ms)
Easy	8	0.75	0.8	0.56	849
Medium	8	0.75	1.6	0.56	564
Hard	4	0.50	2.5	0.44	889
Overall	20	0.70	1.4	0.54	743

Table 9: DAG planner pilot evaluation. Step Acc.: fraction of queries where predicted step count matches ground truth. Edges/q: average dependency edges detected per query. Coherence: functional coherence (fraction of adjacent skill pairs with compatible I/O categories). 80% of queries have at least one detected dependency edge. Median latency is 568ms (p95: 1,261ms). Latency variation across difficulty levels reflects small sample sizes ($n=4-8$) rather than a systematic trend.

F SAD Prompt Templates

System prompt (shared by vanilla and SAD).

```
You are a task decomposition
assistant. Given a complex
user query, break it down into
atomic sub-tasks, each requiring
exactly one tool or skill.
Output a JSON array of strings.
Each string should be a concise,
actionable sub-task description.
```

Vanilla user prompt.

```
Decompose the following query
into atomic sub-tasks:
{query}
```

SAD user prompt (Pass 2).

```
Decompose the following query
into atomic sub-tasks.
Available skills that may be
relevant: {hint_list}
Query: {query}
```

where $\{\text{hint_list}\}$ is a comma-separated list of the top- H skill names retrieved in Pass 1 (see Algorithm 1).